Routing Algorithms and Protocols: Dijkstra, Bellman-Ford, Count to Infinity, and How OSPF and BGP Avoid Routing Loops

for Computer Networks Laboratory

Name: Subham Bhat

Class: BCSE III

Roll: 002410501020

Group: A1

College: Jadavpur University

Contents

1 Introduction **3**

1.1 What Was Built 3

2 The Packets **4**

3 Architecture **5**

4 Dijkstra's Algorithm (Link State) **5**

5 Bellman–Ford (Distance Vector) **7**

5.1 Good News and Bad News 7

5.2 Count to Infinity with Two Routers 8

5.3 Three Routers Defeat Split Horizon and Poison Reverse 8

6 The Same Failure with OSPF and BGP **9**

6.1 OSPF 9

6.2 BGP 10

7 Verification and Results **11**

8 Conclusion **12**

AI Acknowledgement **13**

1 Introduction

Routers build their forwarding tables with a routing algorithm. Two families dominate inside an organisation, and a third connects organisations:

- **Link state** (OSPF, RFC 2328): every router floods the state of its own links to all others, so every router holds the same map and computes shortest paths with **Dijkstra's algorithm**.
- **Distance vector** (RIP, RFC 2453): a router knows only its neighbours and tells them its current distance to every destination. Each router applies the **Bellman–Ford** equation

$$D_x(y) = \min_{v \in \text{neighbours}(x)} \{\, c(x,v) + D_v(y) \,\}.$$

  Good news spreads quickly, but bad news spreads slowly: after a failure, routers can keep using each other's stale distances and **count to infinity** (16 in RIP) while packets loop between them. *Split horizon* and *poison reverse* fix loops between two routers but not loops through three.
- **Path vector** (BGP-4, RFC 4271), used between autonomous systems (ASes): every advertisement carries the full **AS_PATH**, and a router rejects any path that already contains its own AS. Loops therefore cannot form.

1.1 What Was Built

This module demonstrates all three families on small networks and puts every routing message on the wire, in pure Java 11 with no external libraries:

1. **Dijkstra** step by step on the textbook's six-router network, recording N', $D(v)$ and $p(v)$ after every step.
2. **Synchronous Bellman–Ford** with RIP's infinity of 16, optional split horizon and poison reverse, on the six-router network (cold start), a three-router network (good and bad news), a line of three routers and a triangle with a stub.
3. **OSPF** reacting to the same failure: new Router-LSAs with higher sequence numbers are flooded, then Dijkstra is rerun.
4. **BGP** reacting to the same failure with AS_PATH loop detection.
5. **Real packets:** every distance vector is sent as a RIPv2 response over UDP 520, every LSA as an OSPF LS Update (with both checksums) carried in UDP 8900, and every BGP OPEN, UPDATE and KEEPALIVE over a real TCP session on port 179, between six loopback addresses 127.0.0.20–25. Wireshark decodes them.
6. **A dashboard** on `http://localhost:8088` designed as a railway signal box, with one lever per experiment and a **Forward** control that advances one round at a time.

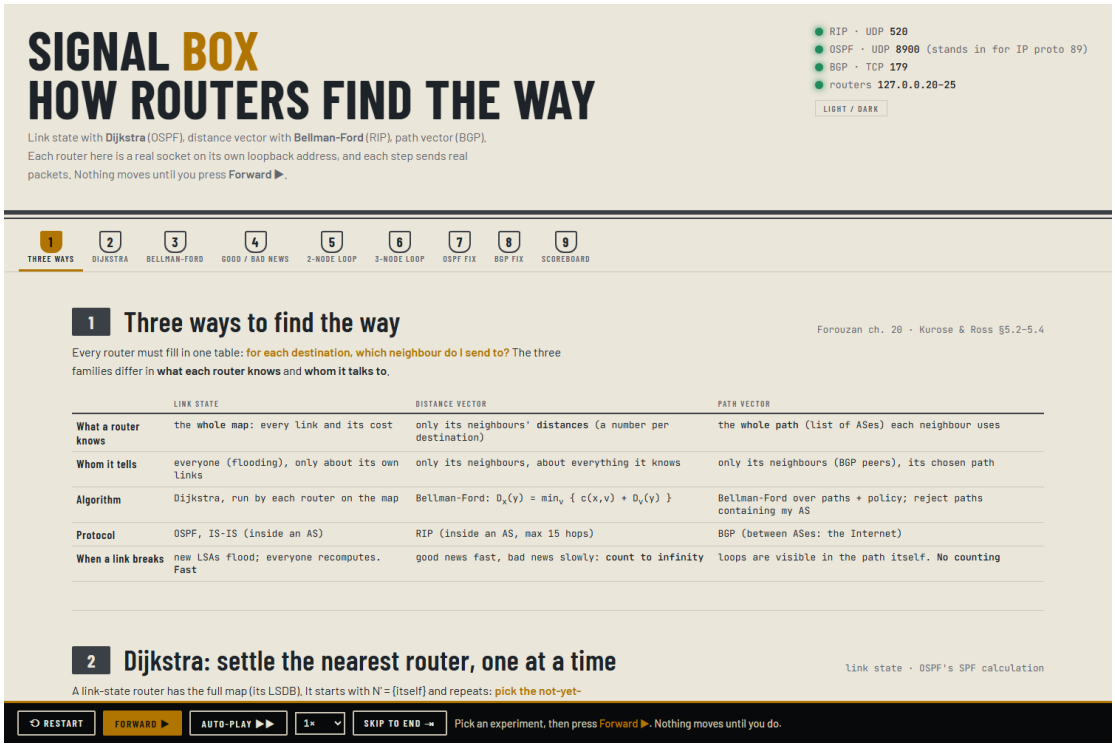


Figure 1: The Routing dashboard.

2 The Packets

Table 1: The three message formats sent on the wire

Protocol	Format
RIPv2 response (UDP 520)	4-byte header (command 2 = response, version 2) followed by 20 bytes per route: address family, route tag, IP address, mask, next hop, metric (1–16, 16 = unreachable).
OSPFv2 LS Update (UDP 8900)	24-byte OSPF header with a one's-complement checksum, then Router-LSAs: a 20-byte LSA header (age, type, link-state ID, advertising router, **sequence number**, Fletcher checksum, length), 4 bytes, and 12 bytes per link.
BGP-4 (TCP 179)	16-byte marker of `0xFF`, length, type. OPEN is 29 bytes and KEEPALIVE 19; UPDATE carries withdrawn routes, path attributes (ORIGIN, **AS_PATH**, NEXT_HOP) and the reachable prefixes.

OSPF normally runs directly on IP (protocol 89), which needs raw sockets; the module carries the identical bytes inside UDP 8900 so that it runs without administrator rights.

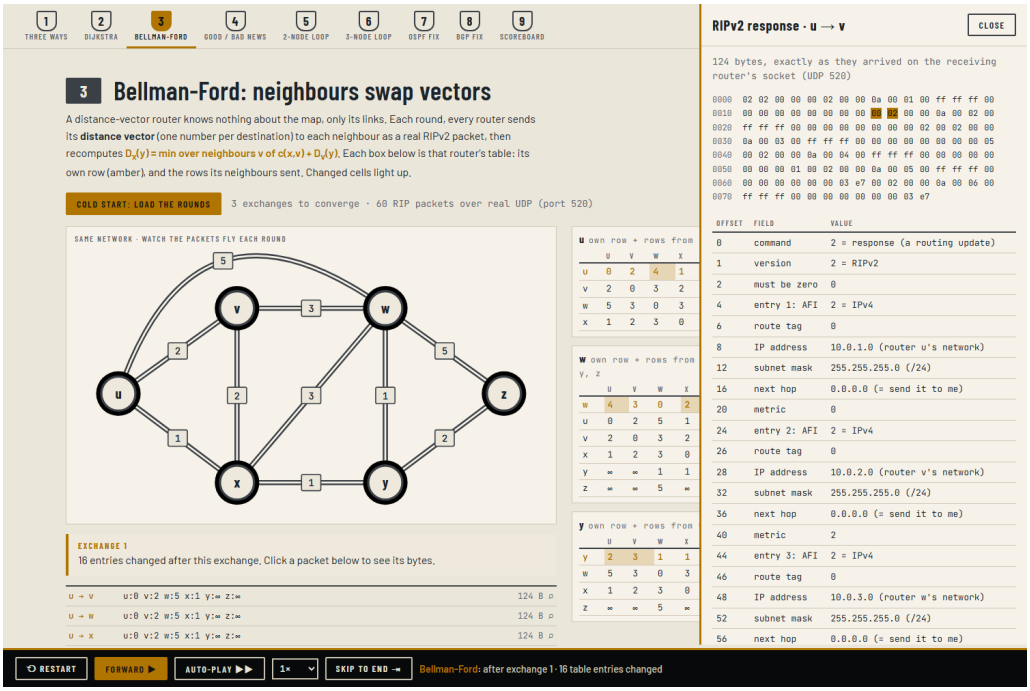


Figure 2: The packet inspector: a RIPv2 response decoded field by field.

3 Architecture

Table 2: Source files

File	Responsibility
`sim/Topo.java`	The test networks: six routers u–z, three routers x–z, a line A–B–C, a triangle A–B–C with stub D, and four ASes 100–400.
`sim/Dijkstra.java`	Dijkstra with the textbook table recorded at each step.
`sim/DistanceVector.java`	Synchronous Bellman–Ford with infinity, split horizon and poison reverse.
`sim/LinkState.java`	OSPF flooding with sequence numbers, then Dijkstra on the LSDB.
`sim/PathVector.java`	BGP route exchange with AS_PATH loop rejection.
`wire/Rip.java`, `Ospf.java`, `Bgp.java`	Encoders and decoders for the three packet formats, including checksums.
`net/Wire.java`	UDP and TCP sockets on 127.0.0.20 + i for router i.
`sim/Scenarios.java`	The experiments, returned to the dashboard round by round.

4 Dijkstra's Algorithm (Link State)

Starting from the source, the algorithm repeatedly settles the unsettled router with the smallest known distance and then checks whether going through it shortens the path to each of its unsettled neighbours.

```
dist[src] = 0; done[src] = true;
for (int v : t.neighbours(src)) { dist[v] = t.cost[src][v]; prev[v] = src; }   // step 0
for (int k = 1; k < n; k++) {
    int w = -1;                                                  // closest unsettled router
```

```
    for (int v = 0; v < n; v++) if (!done[v] && dist[v] < Topo.NONE && (w < 0 || dist[v] < dist[w])) w = v;
    if (w < 0) break;                                   // the rest is unreachable
    done[w] = true;                                     // add w to N'
    for (int v : t.neighbours(w)) {
        if (done[v]) continue;
        int alt = dist[w] + t.cost[w][v];
        if (alt < dist[v]) { dist[v] = alt; prev[v] = w; }      // a shorter path through w
    }
}
```

Listing 1: The main loop of `sim/Dijkstra.java`.

On the textbook network (links u–v 2, u–x 1, u–w 5, v–x 2, v–w 3, x–w 3, x–y 1, y–w 1, y–z 2, w–z 5) from source u, the recorded table is:

Table 3: Dijkstra from u, as produced by the module (∞ = not yet reachable)

Step	N'	$D(v), p(v)$	$D(w), p(w)$	$D(x), p(x)$	$D(y), p(y)$	$D(z), p(z)$
0	u	2,u	5,u	1,u	∞	∞
1	ux	2,u	4,x		2,x	∞
2	uxv		4,x		2,x	∞
3	uxvy		3,y			4,y
4	uxvyw					4,y
5	uxvywz					

The result is $D(v) = 2$, $D(x) = 1$, $D(y) = 2$, $D(w) = 3$, $D(z) = 4$, and u's forwarding table sends traffic for v directly to v and for every other router to x.

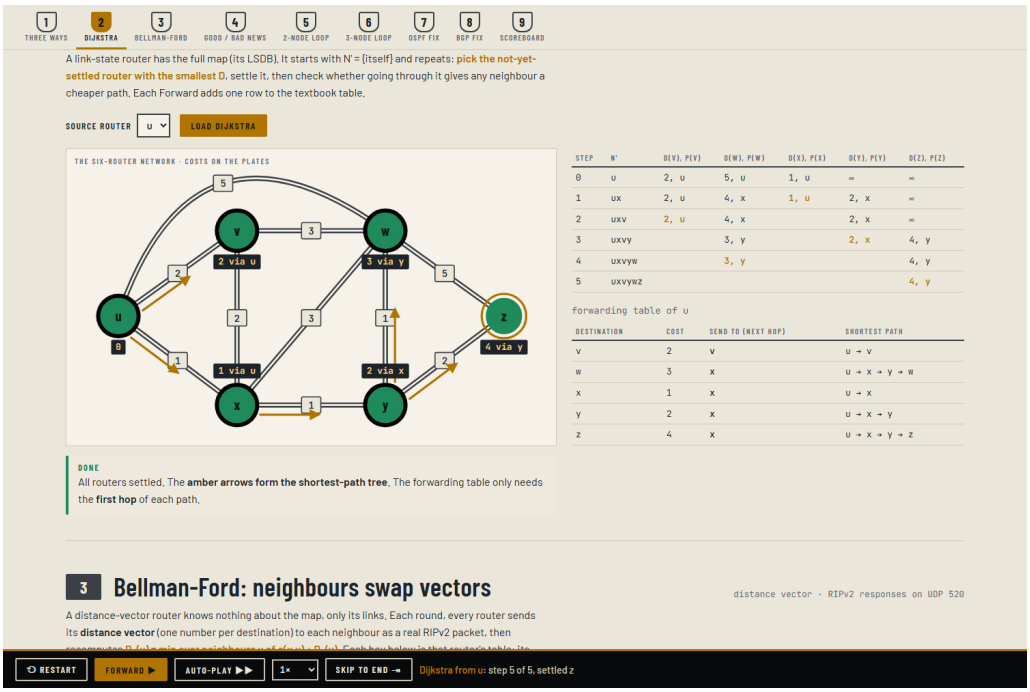


Figure 3: Dijkstra finished: the shortest-path tree from u and its forwarding table.

5 Bellman–Ford (Distance Vector)

In every exchange each router sends its whole distance vector to each neighbour as a RIPv2 response; then every router recomputes its table from the vectors it last heard. Split horizon omits the routes whose next hop is the neighbour being told; poison reverse sends them with metric ∞ instead.

```
1  private List<Rip.Entry> advert(int x, int v, List<Integer> omitted, List<Integer> poisoned)
       {
2      List<Rip.Entry> e = new ArrayList<>();
3      for (int d = 0; d < n; d++) {
4          if (d != x && next[x][d] == v && split) {                    // this route goes
       through v
5              if (poison) { e.add(new Rip.Entry(d, inf)); poisoned.add(d); }   // poison
       reverse: say "infinity"
6              else omitted.add(d);                                     // split horizon: say
       nothing
7              continue;
8          }
9          e.add(new Rip.Entry(d, D[x][d]));
10     }
11     return e;
12 }
```

Listing 2: What router x tells neighbour v (`sim/DistanceVector.java`).

Cold start. On the six-router network, starting from direct links only, 16 table entries changed in the first exchange, 6 in the second and none in the third: converged in 3 exchanges with 60 RIP packets, and every distance equals Dijkstra's.

5.1 Good News and Bad News

Three routers x, y, z with links x–y 4, x–z 50, y–z 1; the table shows each router's distance to x and its next hop.

- **x–y drops from 4 to 1**: converged in 2 exchanges.
- **x–y rises from 4 to 60**: y recomputes using z's old distance 5 (which goes through y itself) and chooses 6 via z. The two now point at each other, and their distances climb by 2 per exchange until z's count exceeds its direct link of 50.

Table 4: Bad news (x–y: 4 → 60), distance to x / next hop

Round	**y**	**z**	
before	4 / x	5 / y	
the change	6 / z	5 / y	loop y ↔ z
exchange 2	6 / z	7 / y	loop
exchange 3	8 / z	7 / y	loop
exchange 4	8 / z	9 / y	loop
…	…	…	
exchange 48	51 / z	50 / x	settled after **47 exchanges**
with poison reverse	60 / x → 51 / z	50 / x	settled in **3 exchanges**, no loop

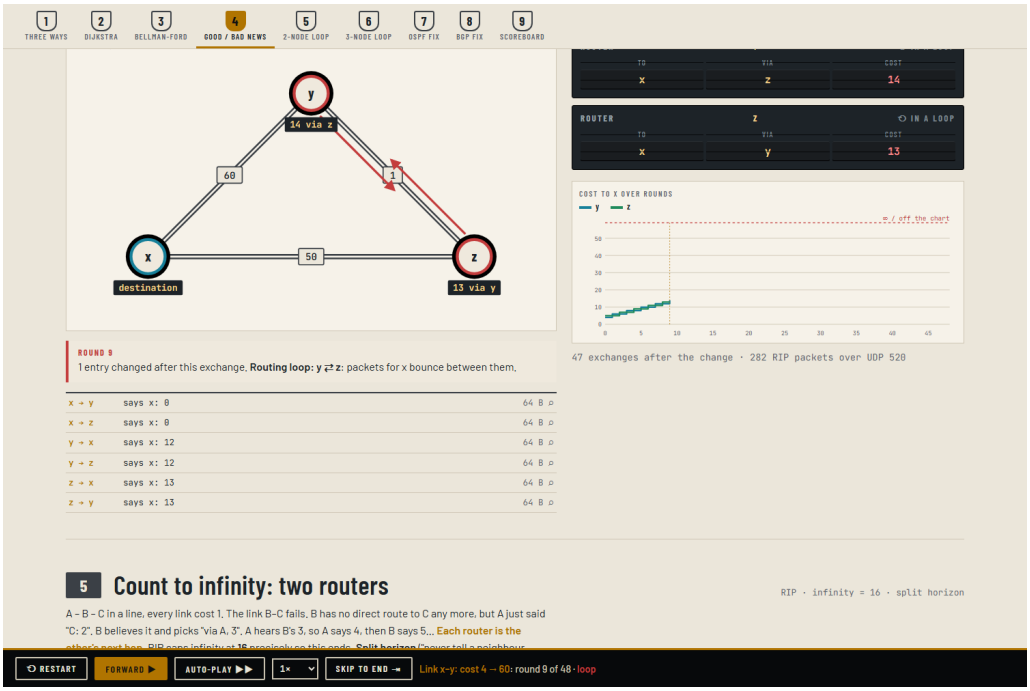


Figure 4: Bad news: y and z point at each other while their distances climb.

5.2 Count to Infinity with Two Routers

A line A–B–C, watching C; the link B–C fails. B has lost C, but A last said "C at 2", so B chooses 3 via A, and the two count up together.

Table 5: Two routers count to infinity (distance to C / next hop)

Round	A	B	
before	2 / B	1 / C	
B–C fails	2 / B	3 / A	loop A ↔ B
exchange 2	4 / B	3 / A	
exchange 3	4 / B	5 / A	
...	...	...	
exchange 16	unreachable	unreachable	after **15 exchanges** (16 = infinity)
with split horizon	unreachable	unreachable	**2 exchanges**: A never told B about C

5.3 Three Routers Defeat Split Horizon and Poison Reverse

A triangle A, B, C with a stub D on C, watching D, with **both** split horizon and poison reverse on; the link C–D fails. B's route to D went through C, not A, so split horizon still lets B tell A "D at 2". A ghost route then circles A → B → C:

Table 6: Triangle with split horizon and poison reverse (distance to D / next hop)

Round	A	B	C	
before	2 / C	2 / C	1 / D	
C–D fails	2 / C	2 / C	unreachable	
exchange 2	3 / B	3 / A	unreachable	loop A ↔ B
exchange 3	unreachable	unreachable	4 / A	
exchange 4	unreachable	5 / C	unreachable	
exchange 5	6 / B	unreachable	unreachable	
...				
exchange 16	unreachable	unreachable	unreachable	**15 exchanges**

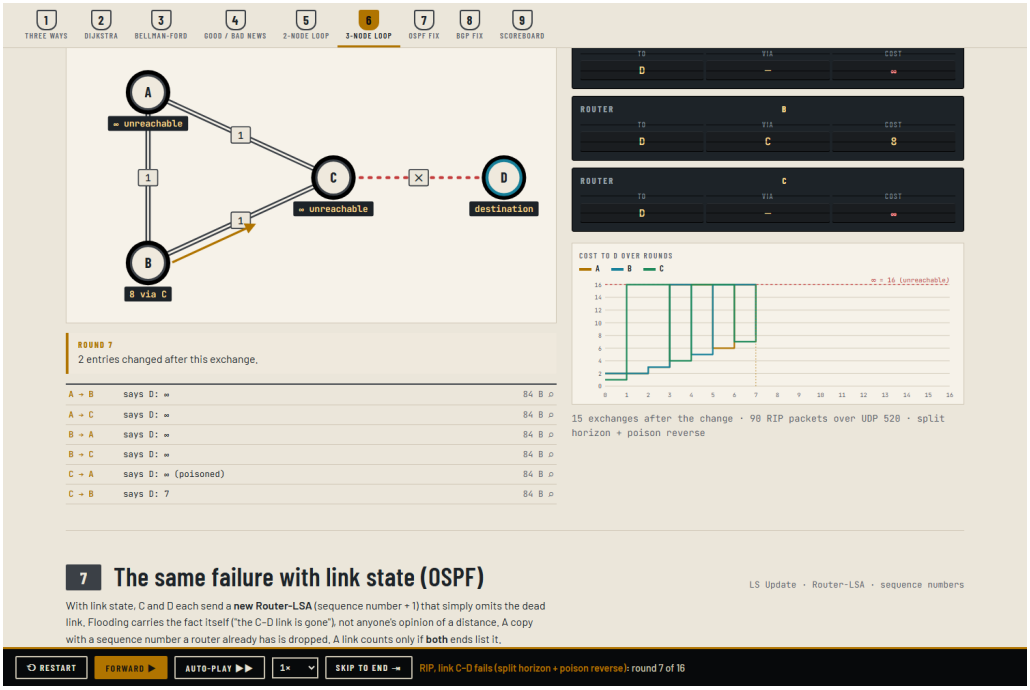


Figure 5: The three-router loop survives split horizon and poison reverse.

6 The Same Failure with OSPF and BGP

6.1 OSPF

When C–D fails, C and D each originate a new Router-LSA listing only their working links, with the sequence number raised from `0x80000001` to `0x80000002`, and flood it. A router installs and forwards an LSA only if its sequence number is newer than the copy it holds; a duplicate is acknowledged and not forwarded, so flooding stops by itself. In the first flooding round 2 LS Updates were delivered and passed on; in the second, the duplicates were dropped. Every router then held the same LSDB and Dijkstra marked D unreachable: **2 rounds, 4 LS Updates, no loop at any time**. Every packet passed both the OSPF one's-complement checksum and the LSA Fletcher checksum.

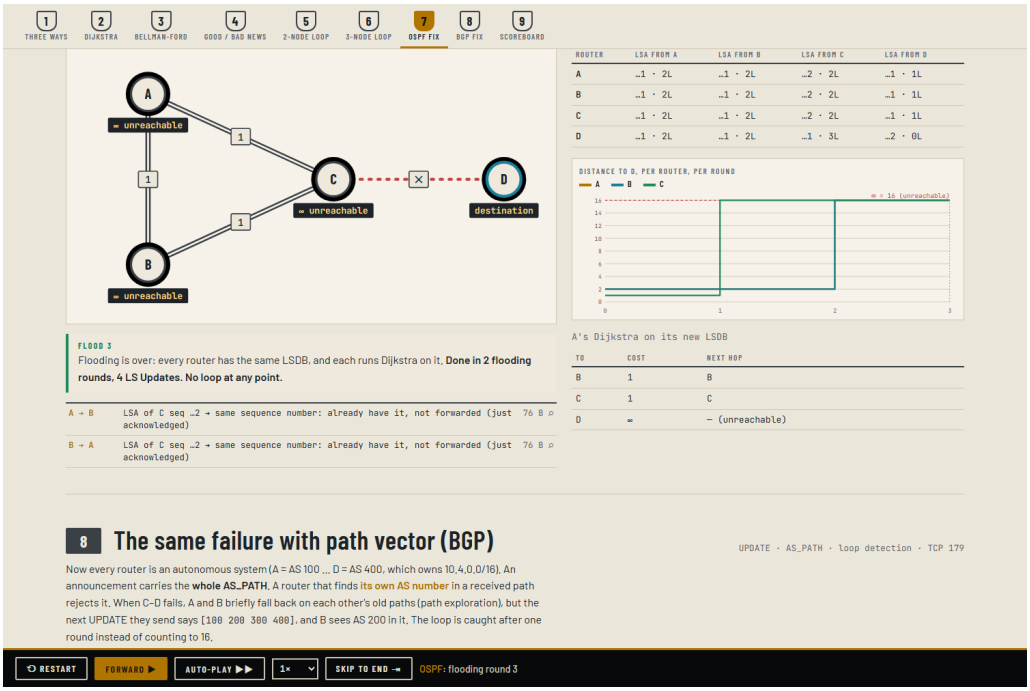


Figure 6: OSPF floods the new LSAs and reruns Dijkstra: no counting.

6.2 BGP

Four ASes in the same triangle-plus-stub shape: A = AS 100, B = AS 200, C = AS 300, and D = AS 400, which owns 10.4.0.0/16. A path is stored only if it does not already contain the receiving router's own AS.

```
if (u.path == null) {                                   // a withdrawal
    ribIn[v][x] = null; ribNote[v][x] = "withdrawn";
} else if (u.path.contains(t.as[v])) {                  // my own AS is already in the path
    ribIn[v][x] = null;
    ribNote[v][x] = "rejected: loop (AS " + t.as[v] + " already in " + u.path + ")";
} else {
    ribIn[v][x] = u.path;                               // store it; best path chosen
    afterwards
}
```

Listing 3: Importing an UPDATE with the AS_PATH loop check (`sim/PathVector.java`).

Table 7: BGP announcement and failure, as recorded

Round	Events
sessions	TCP to port 179 on every link; OPEN and KEEPALIVE each way; all sessions Established
1	D → C: [400]. C stores it.
2	C → A and B: [300 400], stored. C → D: **rejected**, D sees its own AS 400.
3	A → B: [100 300 400], B → A: [200 300 400], kept as backups. Both offers to C are **rejected** (AS 300).
4	Converged: A and B reach 10.4.0.0/16 via [300 400].
C–D fails	The TCP session closes; C drops every route learned over it and withdraws.
6–7	A and B fall back on their backups, which go through C; they offer each other [100 200 300 400] and [200 100 300 400], and each is **rejected: loop**.
8–9	Withdrawals; converged. Only D has a route: no counting and no stale loop, **3 rounds after the failure**.

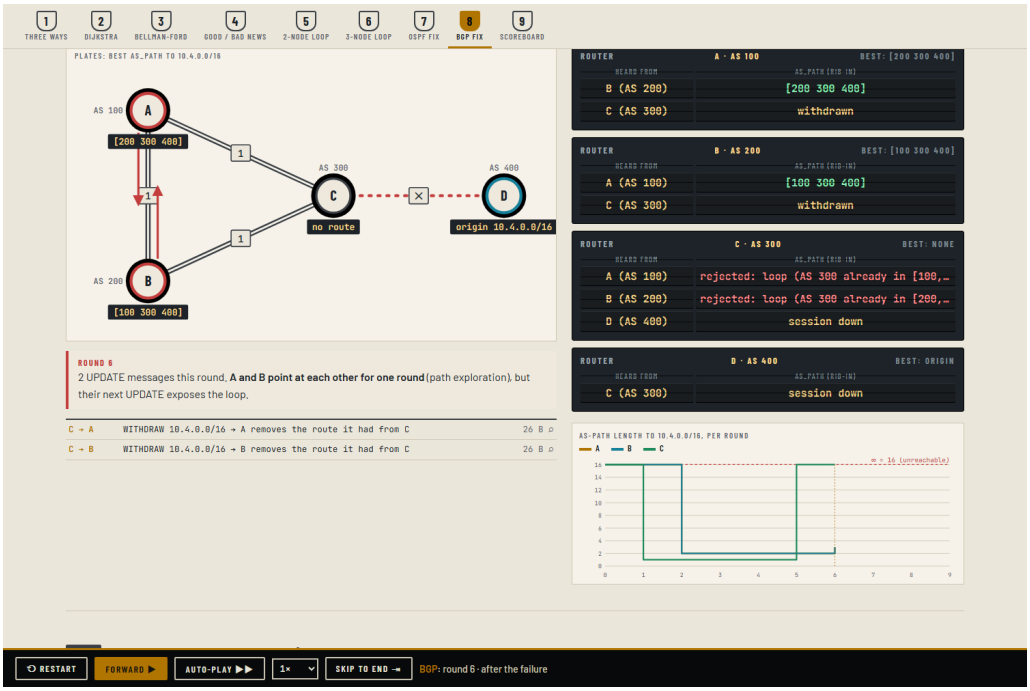


Figure 7: BGP rejects a path that contains the router's own AS.

7 Verification and Results

```
cd Routing
run.bat                  # dashboard (http://localhost:8088); routers on 127.0.0.20 - 127.0.0.25
run.bat --verify         # 30 automated checks
python interop_test.py   # an independent Python decoder checks our RIP, OSPF and BGP bytes (15 checks)
# Wireshark: capture on the loopback adapter, display filter  rip || bgp
```

Listing 4: Running the lab, the checks and the independent decoder.

Table 8: Scoreboard for the triangle-plus-stub failure

Protocol	Rounds after the failure	Loop?
RIP, no split horizon	15	yes
RIP + split horizon + poison reverse	15	yes (3-router loop)
OSPF (link state)	2	no
BGP (path vector)	3	no (own-AS paths rejected)

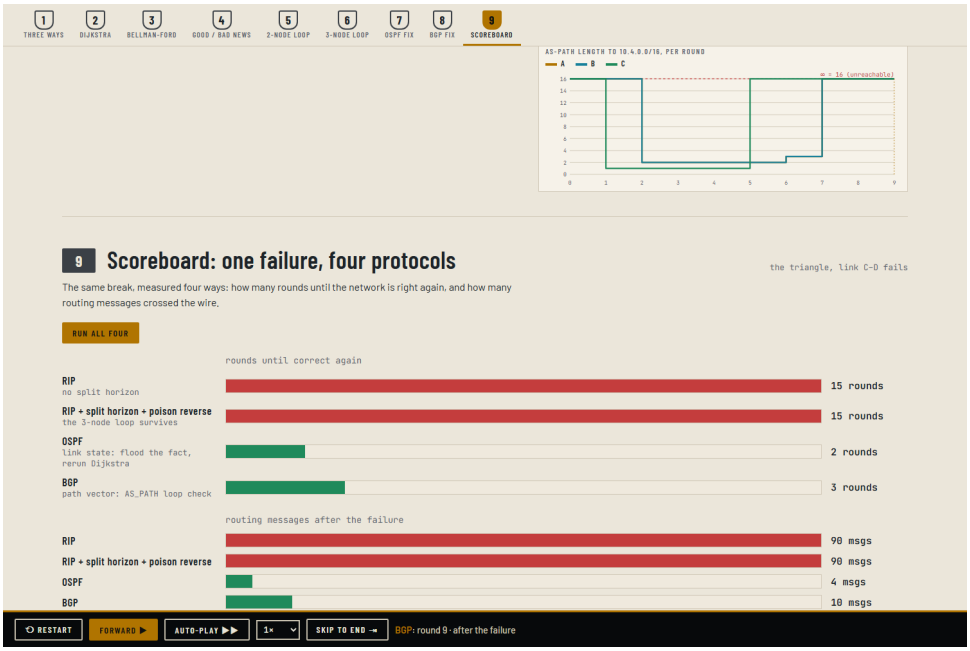


Figure 8: The scoreboard.

Table 9: Selected verification checks (all 30 passed)

Dijkstra from u: $D(v) = 2$, $D(w) = 3$, $D(x) = 1$, $D(y) = 2$, $D(z) = 4$; forwarding table v via v, all others via x; path u x y z
Cold start converges and every distance equals Dijkstra's; every vector crossed a real UDP socket as a RIPv2 response
Good news converges in ≤ 3 rounds; bad news bounces for ≥ 40 rounds with a y $\leftrightarrow$ z loop, then settles at 50 / 51
Poison reverse fixes the three-router cost change in ≤ 3 rounds
Two routers count to 16 and point at each other; split horizon gives up at once
Three routers: split horizon + poison reverse still count to 16
OSPF flooding ends within 3 rounds; all checksums verify; D unreachable with no counting
BGP converges after the failure in ≤ 4 rounds; a router rejected a path containing its own AS; only D keeps a route
RIPv2, Router-LSA, OSPF and BGP (UPDATE, withdrawal, OPEN 29 bytes, KEEPALIVE 19) encodings verified byte by byte

An independent Python decoder, written separately from the Java code, parsed the RIP, OSPF and BGP bytes produced by the module and agreed on every field (15 checks). A loopback capture (`docs/rip_bgp_loopback_capture.pcapng`) shows Wireshark decoding the RIP responses with metric 16 and the BGP UPDATEs with their AS_PATH.

8 Conclusion

1. **Dijkstra and Bellman–Ford reach the same answer by different routes.** The cold-start distance vectors converged to exactly Dijkstra's distances, but distance vector needs only neighbours' vectors while link state needs the whole map.

2. **Distance vector is fragile after failures.** Bad news took 47 exchanges on three routers, and two routers counted to infinity in 15. Split horizon and poison reverse helped only when the loop involved two routers; a three-router loop still counted to 16.

3. **More information removes the loops.** OSPF floods the changed link and recomputes from a complete map (2 rounds), and BGP's AS_PATH lets each router recognise a path through itself (3 rounds). Neither counted to infinity.

AI Acknowledgement

Artificial Intelligence tools were used as an aid during this project. Specifically, AI assistance was used for:

- **Protocol cross-checking:** comparing message formats, reply codes, state machines and byte offsets against the RFCs cited in this report.
- **Code review and testing:** suggesting edge cases for the automated verification suite and reviewing concurrency and error handling.
- **Writing and visualization:** drafting, structuring and refining the explanations, tables, diagrams and LaTeX source of this report.

All protocol implementations, Java source code, experiments, measurements and final conclusions were run, reviewed and verified by the author, who takes full responsibility for the content of this report.